

HAFTA

02

HTTP Protokolü

Geçen hafta iki makinenin birbirini nasıl bulduğunu gördük. Bu hafta buluştuktan sonra ne söylediklerine bakıyoruz — kelimesi kelimesine.

HAFTANIN SORUSU

*Tarayıcı ile sunucu hangi bilgileri
gönderiyor?*

Bu haftanın hedefleri

Her başlığın derinliği aynı değil

✓ MUTLAKA BİL

HTTP istek ve yanıtının yapısını; metotların ve durum kodlarının anlamını açıkla.

>_ DERSTE UYGULA

curl ve Network sekmesiyle gerçek bir HTTP alışverişini incele.

🔍 MERAK EDİYORSAN İNCELE

TCP el sıkışması, ayrıntılı HTTP başlıkları, TLS ve HTTP sürümleri.

Kampüs panosu: iki cümlelik bir konuşma

```
GET /pano HTTP/1.1      <- tarayıcı ne istediğini söylüyor
Host: ornek.edu.tr

HTTP/1.1 200 OK         <- sunucu ne olduğunu söylüyor
Content-Type: text/html
```

Bu dört satır, iki makine arasında gidip gelen metnin tamamıdır. İlk satır niyeti söyler, sonraki satırlar ayrıntıyı taşır. Bu hafta her satırın ne işe yaradığını tek tek göreceğiz.

SORU

Sunucu 404 döndürdüyse, bu konuşmanın neresine kadar her şey yolunda gitmiş demektir?

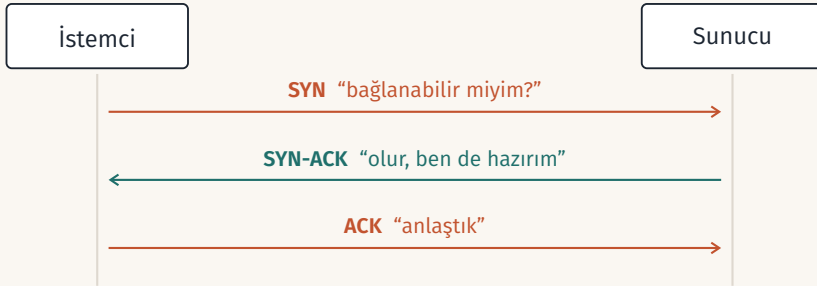
1

Önce bağlantı

Bu bölümde HTTP/1.1 ve HTTP/2 için TCP bağlantısını inceliyoruz.

Üç adımlı el sıkışma

MERAK EDİYORSAN İNCELE



Bu şema, yeni bir TCP bağlantısının **temel el sıkışmasını** gösterir. Uzak sunucularda gecikmenin bir kısmı buradan gelir.

HTTP nerede duruyor?

HTTP — ne söyleneceği

TCP — eksiksiz iletim

IP — hangi makineye

Ethernet / Wi-Fi — kablodan nasıl

Geçen hafta bu dört katmanı aşağıdan yukarı kurmuştuk. Bu hafta yalnızca **en üstteki kutunun içine** bakıyoruz. HTTP/1.1 mesajları okunabilir metindir — bu yüzden elle okuyabiliyoruz. HTTP/2 ve HTTP/3 aynı anlamları ikili çerçevelerle taşır.

2

İstek

Tarayıcının sunucuya söylediği her şey üç parçadan oluşur.

Bir HTTP isteğinin anatomisi

```
GET /pano?bolum=bm HTTP/1.1      <- istek satırı
Host: ornek.edu.tr                <- başlık
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive

                                   <- boş satır
                                   <- gövde (bu GET örneğinde yok)
```

- **İstek satırı**: metot, yol, protokol sürümü
- **Başlıklar**: isteğe eşlik eden bilgiler, her biri Ad: Değer
- **Boş satır**: başlıkların bittiğini söyler — kritik ayraç

HTTP metotları işlemin amacını belirtir

METOD	NE DEMEK	GÜVENLİ	İDEMPOTENT
GET	Kaynağı getir	evet	evet
POST	Veriyi işleme sun	hayır	hayır
PUT	Kaynağı oluştur / değiştir	hayır	evet
PATCH	Kaydın bir kısmını değiştir	hayır	garanti yok
DELETE	Kaydı sil	hayır	evet

İKİ TERİM

Güvenli: sunucuda bir şeyi değiştirmeyi amaçlamaz — tarayıcı ve arama motorları bu yüzden izinsiz tekrarlayabilir. **İdempotent:** aynı isteği bir kez de on kez de göndererseniz sunucuda oluşan sonuç aynıdır; dönen durum kodu farklı olabilir.

Metot bir sözleşmedir, bir garanti değil

GET ile gelen bir isteğin veriyi silmesini engelleyen hiçbir teknik mekanizma yoktur.

Metotların tanımlanmış anlamları vardır. Bu anlamlara uymak sunucuyu yazan kişinin sorumluluğudur. Uymayan API'ler tarayıcı önbelleği, arama motoru botları ve ağ katmanı tarafından **beklenmedik biçimde** cezalandırılır.

→ **WP-II** DRF'te `ViewSet`'ler bu sözleşmeyi sizin için doğru kurar — ama nedenini bilmek gerekir.

Host: hangi siteyi istediğinizi söyler — tek IP’de yüzlerce site barınabildiği için zorunludur

User-Agent: tarayıcı ve işletim sistemi bilgisi

Accept: “hangi biçimi anlarım” — text/html, application/json

Accept-Language: dil tercihi

Authorization: kimlik bilgisi — 11. haftada geri gelecek

Content-Type: gövdedeki verinin biçimi — POST’ta önemli

NOT

Başlıklar **genişletilebilir**: protokol yeni bir başlık eklendiğinde bozulmaz. HTTP’nin otuz yıldır ayakta kalmasının sebeplerinden biri budur.

Sunucuyla elle konuşalım

>_ CANLI DEMO

curl -v ile ham isteği ve ham yanıtı birlikte okuyalım.

```
$ curl --http1.1 -v https://example.com
> GET / HTTP/1.1          <- bizim gönderdiğimiz
> Host: example.com
>
< HTTP/1.1 200 OK          <- sunucunun cevabı
< Content-Type: text/html; charset=UTF-8
```

> bizim gönderdiğimiz, < sunucunun döndüğü satırlardır.

3

Yanıt

Sunucunun cevabı da aynı üç parçadan oluşur — ilk satırı hariç.

Bir HTTP yanıtının anatomisi

HTTP/1.1 200 OK	<- durum satırı
Content-Type: text/html; charset=UTF-8	<- başlık
Content-Length: 1256	
Date: Mon, 15 Sep 2025 10:22:31 GMT	
	<- boş satır
<!doctype html>	<- gövde
<html> ... </html>	

Tek fark ilk satırda: istek **ne istediğini**, yanıt **ne olduğunu** söyler. Gerisi aynı yapıdır — başlıklar, boş satır, gövde.

Durum kodları — beş aile

1xx Bilgi nadir	2xx Başarılı 200, 201, 204	3xx Yönlendirme 301, 302, 304	4xx İstemci hatası 400, 401, 403, 404	5xx Sunucu hatası 500, 502, 503
------------------------------	---	--	---	--

İlk rakam yanıtın **sınıfını** söyler: 4xx isteğin karşılanamadığını, 5xx sunucu tarafında bir sorun olduğunu belirtir. Aynı ailedeki her kod aynı işi yapmaz — 304, 3xx ailesinde olmasına rağmen başka adrese göndermez; önbellekteki kopyanın hâlâ geçerli olduğunu söyler.

Karıştırılan çiftler

401 UNAUTHORIZED

“Kim olduğumu bilmiyorum.”

Giriş yapılmamış veya kimlik bilgisi geçersiz. Geçerli kimlik bilgisi gerekir; yetki ayrıca denetlenir.

Doğru tepki: giriş sayfasına yönlendirir.

403 FORBIDDEN

“Bu isteğe izin vermiyorum.”

Sunucu isteği anladı, karşılamayı reddediyor. Kimliğin bilinmesi zorunlu değildir.

Doğru tepki: “yetkiniz yok” de.

BİR ÇİFT DAHA

301 kalıcı taşınma (tarayıcı ve arama motoru bunu ezberler), **302** geçici yönlendirme. Yanlış seçim geri alınması zor sonuçlar doğurur.

Content-Type: gövdenin biçimi — tarayıcı buna göre davranır

Content-Length: gövdenin bayt cinsinden uzunluğu

Location: yönlendirmede gidilecek adres — 3xx ile birlikte

Set-Cookie: tarayıcıya bir şey hatırlatma talimatı

Cache-Control: bu yanıt ne kadar saklanabilir

⚠ 11. HAFTA İLE BAĞLANTI

Son iki başlık, **durumsuz** bir protokolün sizi nasıl hatırladığının ve aynı veriyi neden tekrar tekrar indirmedığınızın cevabıdır.

Hataları kasten üretelim

>_ CANLI DEMO

Verilen test adreslerinden gelen durum satırlarını karşılaştıralım; sonuçları tahmin edip ölçelim.

```
$ curl -s -o /dev/null -w "%{http_code}\n" https://example.com
200
```

```
$ curl -s -o /dev/null -w "%{http_code}\n" https://example.com/yokboyle
404
```

Tarayıcının gösterdiği “sayfa bulunamadı” ekranı bir **tasarım tercihidir**; protokol düzeyinde durum kodu 404, açıklama ise yanıt gövdesidir.

Birlikte çözelim: aynı isteği iki kez gönderelim

```
$ curl -s -o /dev/null -w "%{http_code}\n" -X DELETE $API/urunler/12  
204
```

```
$ curl -s -o /dev/null -w "%{http_code}\n" -X DELETE $API/urunler/12  
404
```

İki çağrı aynı, iki yanıt farklı. Peki DELETE hâlâ idempotent mi?

NOT

Evet. İdempotentlik **durum koduna değil, sunucuda oluşan sonuca** bakar: her iki çağrıdan sonra da 12 numaralı kayıt yoktur. En sık karıştırılan nokta tam burasıdır.

Sıra sizde

🕒 ALIŞTIRMA · 20 dk

Size verilen dört adrese `curl` ile istek atın ve şu tabloyu doldurun:

1. Durum kodu ve ait olduğu aile
2. Content-Type başlığının değeri
3. Yanıtta Location başlığı var mı — varsa nereye yönlendiriyor?

Ardından aynı adreslerden birini tarayıcıda açıp **F12 → Network** sekmesinde aynı bilgileri bulun. **İki araç da aynı şeyi gösteriyor mu?**

4

Güvenlik ve evrim

Aynı protokol, otuz yılda üç kez hızlandı ve bir kez şifrelendi.

HTTPS — şifreli bağlantı üzerinden HTTP

MERAK EDİYORSAN İNCELE



- TLS **araya giren birinin** veriyi okumasını ve değiştirmesini engeller.
- Sertifika doğrulaması, bağlanılan alan adı ile sunucunun anahtarını ilişkilendirir.
- HTTP'nin kendisi hiç değişmedi — yalnızca altına bir katman eklendi.

→ **WP-II** HTTPS, WP-II'de canlıya çıkarken zorunlu olacak. Bazı tarayıcı özellikleri onsuz hiç çalışmaz.

HTTP/1.1 Bir bağlantıda yanıtlar sıralıdır; birden çok bağlantı kullanılabilir.

HTTP/2 Tek bağlantı üzerinde paralel istekler; başlık sıkıştırma.

HTTP/3 TCP yerine UDP tabanlı QUIC; bağlantı kurulumu daha hızlı.

ORTAK NOKTA

Temel **metot ve durum kodu anlamları** korunur. Mesajın ağda kodlanması ve bağlantı yönetimi farklıdır.

HTTP'nin bir sınırı var: konuşmayı **hep istemci başlatır**. Sunucunun kendiliğinden mesaj göndermesi gerektiğinde **WebSocket** devreye girer — canlı sohbet, bildirim, oyun.

Üç soru

Her yanıtı bir gerekçeyle birlikte verin

1. İstek başlıkları ile gövde birbirinden nasıl ayrılır?
2. POST yalnızca yeni kayıt oluşturmak için midir?
3. “Güvenli metot” ile “şifreli bağlantı” aynı şey midir?

Haftanın sorusunun cevabı

İSTEK

GET /pano HTTP/1.1
Host: ornek.edu.tr
Accept: text/html
metot · yol · başlıklar · gövde



YANIT

HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256
durum · başlıklar · gövde

Burada **HTTP/1.1 metin gösterimini** okuduk. HTTP/2 ve HTTP/3 aynı temel anlamları ikili çerçevelerle taşır.

Nerede kaldık?

İki hafta sonra: protokol katmanı da yerine oturdu

01 Ağ ve DNS	02 HTTP	03 HTML ve DOM	04 CSS
05 Formlar	06 Git ve yayın	07 JavaScript	08 DOM ve olaylar
09 Asenkron ve JSON	10 Fetch ve API	11 Durum ve önbellek	12 Backend
13 Veritabanı	14 Güvenlik		

Kiremit çerçeve: işlendi. Gri çerçeve: sonraki haftalar.

Sunucu bize bir metin gönderdi. Tarayıcı o metni **nasıl ekrana çeviriyor?**

HTML'in yapısını, tarayıcının onu ayrıştırma sürecini ve ortaya çıkan **DOM ağacını** inceleyeceğiz.

→ WEB PROGRAMLAMA II

Bu haftanın metot ve durum kodu bilgisi, WP-II'nin 6. haftasında DRF ile endpoint yazarken doğrudan kullanılacak.